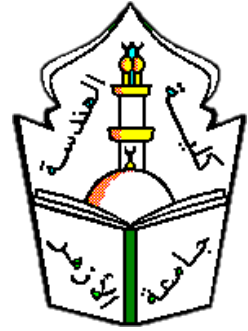




Al-Azhar University
Faculty of Engineering
Systems and Computers Engineering
Department



Smart AI Library - Intelligent Book Recommendation System (Learnova)

Prepared by:

Reham Ehab El-Sayed Ahmed Barakat

Mariam Mohamed Abdullah Mohamed

Norhan Abdelrahman Fathy Salam

Hend Mohamed Ahmed Abdel Motaleb

Wafaa Yousry Ahmed Kohia

Salma Ibrahim Ali

Supervised by:

Dr. Sayed Noah

July 2026

Acknowledgments

In the name of Allah, the Most Gracious, the Most Merciful.

All praise and thanks are due to Allah, the Lord of the worlds, for His countless blessings and for granting us the strength and knowledge to complete this project. Without His guidance and mercy, none of this would have been possible.

We express our deepest gratitude to our beloved families for their unwavering support, prayers, and encouragement throughout our academic journey. Their love and faith have been a constant source of motivation and strength.

We are profoundly grateful to our esteemed supervisor, **Dr. Sayed Noah**, for his invaluable guidance, patience, and insightful feedback. His expertise and dedication were instrumental in shaping the quality and direction of this work.

We also extend our sincere thanks to our teachers and mentors who have imparted wisdom and knowledge throughout our studies. Their efforts have contributed significantly to our personal and academic growth.

May Allah reward everyone who supported us in this endeavor and bless them abundantly.

Alhamdulillah.

Abstract

In the contemporary digital learning landscape, the challenge of delivering personalized, accessible, and time-efficient education has become increasingly critical. Modern learners face a fundamental paradox: unprecedented access to knowledge paired with diminishing time to engage with it. This project presents **Learnova** — an AI-powered Audiobook and E-Learning platform designed to bridge the gap between fragmented audiobook services and structured educational environments through intelligent automation, audio-first delivery, and interactive course support.

The platform enables users to transform PDF books and educational materials into immersive, high-quality audio experiences via a state-of-the-art Text-to-Speech (TTS) pipeline, delivering a Spotify-inspired listening interface with persistent Mini Player support, adjustable playback speed, and chapter-level navigation. In parallel, Learnova provides a structured course management system featuring video lessons with AI-generated transcripts, enabling learners to search, review, and interact with course content through natural language. The **TranscribeAllLessons** pipeline automatically processes video lessons through an AI transcription service, making all course content fully searchable and accessible.

Beyond audio and course delivery, Learnova integrates a suite of six Python AI microservices covering the full intelligent library pipeline: semantic vectorisation using **all-mpnet-base-v2** embeddings stored in Pinecone, a FLAN-T5-based map-reduce summarisation service, a Retrieval-Augmented Generation (RAG) Q&A service powered by GPT-4o-mini with Server-Sent Events streaming and page-level citation, a hybrid recommendation engine combining Neural Collaborative Filtering (NCF) with content-based filtering, and a semantic search service with intent-aware re-ranking. A context-aware conversational chatbot provides on-demand assistance grounded in book content, ensuring each user receives accurate, citation-backed answers.

The system is built on a modern cross-platform mobile architecture using **Flutter** and **Dart** for the frontend with BLoC state management, backed by a **PHP Laravel 12** RESTful API and **MySQL 8.0** database. Laravel acts as a central AI proxy gateway, forwarding requests to the appropriate microservice and degrading gracefully on service unavailability. An administrative panel built with FilamentPHP 3 provides full content management, user role control, and AI metadata oversight. Secure authentication is

handled through JWT and Laravel Sanctum, with support for OTP-based password reset and social login via Google and Facebook OAuth.

This project demonstrates the practical feasibility of unifying audio-first consumption, structured course progression, and a full AI knowledge pipeline within a single cohesive platform, establishing Learnova as a foundational model for next-generation educational systems that prioritize accessibility, personalization, and learner autonomy.

Contents

Acknowledgments	i
Abstract	ii
1 Introduction	1
1.1 Overview	2
1.2 Background of the Project	2
1.3 Target Users	3
1.4 Motivation	3
1.5 Expected Benefits	4
1.6 Problem Statement	4
1.7 Project Contributions	5
1.8 Objectives of the Project	6
1.9 Main Features	6
1.10 Scope and Limitations	7
1.10.1 Scope	7
1.10.2 Limitations	7
1.11 System Architecture Overview	8
1.12 Methodology	8
1.12.1 Main Phases	8
1.12.2 Tools and Technologies	9
1.13 Project Management	9
1.13.1 Timeline and Milestones	10
1.13.2 Team Roles and Responsibilities	10
Chapter Summary	10
2 Literature Review	12
2.1 Overview	13
2.2 Existing E-Learning and Audiobook Platforms	13
2.2.1 Popular Audiobook Platforms	13

2.2.2	Popular E-Learning Platforms	13
2.3	Comparison of Traditional and AI-Powered Platforms	14
2.4	Applications of AI in Education	14
2.5	Recommendation Systems in E-Learning	14
2.6	Retrieval-Augmented Generation (RAG) and LangChain	15
2.7	Text-to-Speech Technologies	16
2.8	Flutter as a Cross-Platform Framework	16
2.9	Related Works	16
2.10	Limitations of Existing Systems	17
	Chapter Summary	18
3	Backend Implementation	19
3.1	Overview	20
3.2	Objectives	20
3.3	Scope	21
3.3.1	Included	21
3.3.2	Excluded	21
3.4	System Architecture	21
3.4.1	AI Proxy Architecture	22
3.5	Database Design	23
3.5.1	users	23
3.5.2	books	23
3.5.3	courses	24
3.5.4	course_videos	24
3.5.5	reviews	24
3.5.6	bookmarks and personal_shelves	25
3.5.7	ai_metadata	25
3.6	API Documentation	25
3.6.1	Authentication Endpoints	26
3.6.2	Books and Discovery Endpoints	26
3.6.3	Courses Endpoints	27
3.6.4	AI Services Endpoints	27
3.7	AI Microservices	28
3.7.1	Courses AI — TranscribeAllLessons	29
3.7.2	AI Service Configuration	29
3.8	Admin Panel (FilamentPHP 3)	30

3.9	Testing	31
3.9.1	Testing Tools	31
3.9.2	Sample Test Cases	32
3.10	Challenges Faced	32
3.11	Future Work	33
	Chapter Summary	34
4	Intelligent Features of the System	35
4.1	Overview	36
4.2	Ingestion Service — Document Processing Pipeline	36
4.2.1	Text Extraction	36
4.2.2	Cleaning and Chunking	36
4.2.3	Kafka Publishing and Status Tracking	37
4.3	Embedding Service — Semantic Vectorisation	37
4.3.1	Design Rationale	37
4.3.2	Model: all-mpnet-base-v2	38
4.3.3	BookEncoder Implementation	38
4.3.4	Kafka-to-Pinecone Pipeline and Fault Tolerance	39
4.4	Search Service — Two-Stage Semantic Retrieval	39
4.4.1	Two-Stage Pipeline	40
4.4.2	Caching and Similar-Book Search	40
4.5	Summarisation Service	41
4.5.1	Model: FLAN-T5-large	41
4.5.2	Map-Reduce Pipeline	41
4.5.3	Mind Map Generation	42
4.6	RAG Q&A Service	42
4.6.1	Architecture	42
4.6.2	Retriever — Similarity Threshold Filtering	43
4.6.3	Chain — Grounded Prompting and Token Budget	43
4.6.4	Streaming — Server-Sent Events (SSE)	44
4.7	Recommendation Service — Hybrid NCF + CBF	44
4.7.1	Neural Collaborative Filtering (NCF)	44
4.7.2	Content-Based Filtering (CBF)	45
4.7.3	Hybrid Fusion and User Tiers	45
4.8	Ethical Considerations	46
4.8.1	Hallucination Mitigation	46
4.8.2	Transparency and Source Attribution	46
4.8.3	Content Scope and Copyright	46

4.8.4 Model Bias and Limitations	47
Chapter Summary	47

List of Figures

3.1	AI Proxy Request Flow in Learnova	22
-----	---	----

List of Tables

1.1	Tools and Technologies Used in Learnova	9
2.1	Comparison of Traditional vs. AI-Powered E-Learning Platforms	14
3.1	System Architecture Layers and Responsibilities	22
3.2	users Table Schema	23
3.3	books Table Schema	23
3.4	courses Table Schema	24
3.5	course_videos Table Schema	24
3.6	reviews Table Schema	24
3.7	bookmarks / personal_shelves Table Schema	25
3.8	ai_metadata Table Schema	25
3.9	Authentication API Endpoints	26
3.10	Books and Discovery API Endpoints	26
3.11	Courses API Endpoints	27
3.12	AI Services API Endpoints	28
3.13	Python AI Microservices Overview	29
3.14	AI Service Configuration Parameters	30
3.15	Admin Panel Sections and Resources	30
3.16	Admin Panel Default Credentials	30
3.17	Testing Tools and Scope	31
3.18	Sample API Test Cases and Expected Results	32
4.1	Ingestion Service Chunking Parameters	37
4.2	Embedding Model Configuration and Design Decisions	38
4.3	Embedding Batch Job Processing Stages	39
4.4	Search Service Two-Stage Pipeline	40
4.5	FLAN-T5 Prompt Configurations by Summary Type	41
4.6	Mind Map Generation Pipeline with Fallback Strategy	42
4.7	RAG Service Component Overview	43
4.8	SSE Protocol Events in the RAG Streaming Endpoint	44
4.9	NCF Model Hyperparameters	45
4.10	Hybrid Fusion Alpha Values by User Tier	46

Chapter 1

Introduction

Contents

1.1	Overview	2
1.2	Background of the Project	2
1.3	Target Users	3
1.4	Motivation	3
1.5	Expected Benefits	4
1.6	Problem Statement	4
1.7	Project Contributions	5
1.8	Objectives of the Project	6
1.9	Main Features	6
1.10	Scope and Limitations	7
1.11	System Architecture Overview	8
1.12	Methodology	8
1.13	Project Management	9

1.1 Overview

The rapid growth of digital information and online learning resources has transformed the way people access knowledge. Thousands of books, research papers, and educational materials are now available in digital formats, providing unprecedented opportunities for learning and self-development. However, the increasing volume of available content has created new challenges in discovering relevant resources and extracting useful knowledge efficiently.

Learnova is an AI-powered Audiobook and E-Learning platform designed to enhance knowledge discovery and reading experiences. The system combines Artificial Intelligence technologies with digital library services to provide users with personalized book recommendations, semantic search capabilities, AI-generated summaries, and intelligent question-answering features.

Unlike traditional digital libraries that primarily focus on storing and retrieving books, Learnova acts as an intelligent reading assistant that helps users find relevant resources, understand content faster, and receive personalized learning experiences. Through its user-friendly mobile application and AI-powered modules, the platform aims to improve accessibility, efficiency, and engagement in digital reading and knowledge exploration.

This chapter introduces the project by discussing its background, target users, motivation, expected benefits, and the problem it aims to solve.

1.2 Background of the Project

Digital libraries have become essential tools for education, research, and knowledge management. Universities, schools, and organizations increasingly rely on digital repositories to provide users with easy access to books, journals, and learning materials. These systems have significantly improved information availability and accessibility compared to traditional physical libraries.

Despite these advancements, many digital library platforms still rely on conventional search mechanisms and static content organization methods. Users often struggle to locate relevant resources among thousands of available documents, especially when search results depend solely on keyword matching. As digital collections continue to expand, the process of finding appropriate books and learning materials becomes increasingly challenging.

At the same time, Artificial Intelligence technologies have revolutionized information retrieval, recommendation systems, and content analysis. AI-powered recommendation engines are widely used in modern platforms to personalize user experiences, while Natural Language Processing (NLP) techniques enable systems to understand content

meaning and provide more accurate search results.

The integration of AI capabilities into digital libraries creates opportunities for developing smarter and more personalized knowledge platforms. By combining recommendation systems, semantic search, summarization, and intelligent question-answering services, Learnova aims to address the limitations of traditional digital libraries and improve the overall reading experience.

1.3 Target Users

Learnova is designed to serve a wide range of users who interact with digital knowledge resources. The primary target users include:

- **Students** seeking quick access to academic books, educational materials, and learning resources.
- **Researchers** who require efficient methods for exploring large collections of documents and extracting relevant information.
- **Educators and instructors** looking for intelligent tools to support teaching and content discovery.
- **General readers** interested in exploring books and receiving personalized recommendations based on their interests.
- **Libraries and educational institutions** seeking modern digital transformation solutions that improve resource accessibility and user engagement.

The platform is designed to accommodate users with varying levels of technical expertise while providing advanced capabilities for academic and professional use.

1.4 Motivation

The development of Learnova is motivated by several challenges faced by users of traditional digital libraries and digital reading platforms.

First, the amount of available digital content continues to grow rapidly, making it difficult for users to discover books and resources that match their interests and needs. Traditional search systems often require users to spend significant time browsing and filtering large collections of content.

Second, most digital libraries rely heavily on keyword-based search methods that may not accurately understand user intent or the contextual meaning of documents. This limitation often leads to irrelevant search results and inefficient information retrieval.

Third, users frequently struggle to determine whether a book or document contains the

information they need without reading large portions of the content. This process can be time-consuming and discouraging.

Additionally, many digital libraries lack personalized recommendation systems that can guide users toward relevant resources based on their preferences, reading history, and interests. Recent advancements in Artificial Intelligence provide an opportunity to overcome these limitations through intelligent recommendations, semantic search, content summarization, and interactive question-answering capabilities.

1.5 Expected Benefits

The implementation of Learnova is expected to provide numerous benefits for users and educational organizations:

- Faster and more accurate discovery of books and learning resources.
- Personalized book recommendations based on user interests, preferences, and reading behavior.
- Improved information retrieval through semantic search technologies.
- Reduced reading time through AI-generated summaries.
- Enhanced understanding of content through intelligent question-answering features.
- Better organization and management of personal reading collections.
- Increased user engagement through personalized and interactive experiences.
- Improved accessibility to knowledge resources for students, researchers, and general readers.
- Support for digital transformation initiatives in educational institutions and libraries.
- A scalable platform capable of supporting future enhancements such as audiobooks and OCR services.

1.6 Problem Statement

Traditional digital libraries provide effective storage and access to digital resources; however, they often lack intelligent mechanisms for personalized content discovery, contextual understanding, and user-centered interaction. Most existing systems depend primarily on keyword-based search techniques, which may fail to capture the actual intent of users or the semantic meaning of documents.

As digital collections continue to grow, users face increasing difficulties in locating relevant books, identifying useful resources, and extracting valuable information efficiently. Furthermore, users often struggle to discover books that align with their interests and learning objectives due to the limited personalization capabilities available in traditional library systems.

The absence of advanced features such as semantic search, intelligent recommendations, automatic summarization, and direct question-answering further reduces the effectiveness of conventional digital libraries and limits user engagement. Consequently, users spend considerable time searching, filtering, and reviewing information manually, which negatively impacts productivity and learning efficiency.

Therefore, there is a need for an intelligent digital library platform that leverages Artificial Intelligence technologies to improve content discovery, resource recommendation, information retrieval, and user interaction. Learnova addresses this need by integrating recommendation systems, semantic search, AI-powered summarization, and intelligent question-answering capabilities into a unified digital knowledge platform.

1.7 Project Contributions

Learnova introduces several innovative contributions that enhance the functionality of traditional digital libraries and improve the overall reading experience:

- Development of an AI-powered digital library that combines intelligent book discovery with personalized reading assistance.
- Integration of a recommendation engine that suggests books and resources based on user preferences, reading history, ratings, and interests.
- Implementation of semantic search capabilities that understand the meaning and context of user queries rather than relying solely on keyword matching.
- AI-powered book summarization that enables users to quickly understand the main ideas and key concepts of books.
- Direct Question-and-Answer functionality that allows users to interact with book content and obtain relevant answers efficiently.
- Development of a modern cross-platform mobile application using **Flutter**, ensuring accessibility across Android and iOS devices.
- Creation of a personalized reading environment through favorites, bookmarks, notes, and personal library management features.
- Transformation of traditional digital libraries into intelligent reading assistants that support knowledge discovery and lifelong learning.

1.8 Objectives of the Project

The primary objective of Learnova is to develop an intelligent digital library platform that enhances knowledge discovery, content understanding, and user engagement through Artificial Intelligence technologies.

The specific objectives of the project are:

- Provide users with centralized and intelligent access to books, references, and educational resources.
- Implement an AI-driven recommendation system based on user preferences and reading behavior.
- Support semantic search and advanced filtering techniques beyond traditional keyword matching.
- Enable users to create personal reading shelves, bookmarks, notes, and favorites.
- Generate AI-powered summaries that reduce reading time and improve content understanding.
- Provide direct question-answering capabilities from book content.
- Deliver a clean, modern, and user-friendly mobile experience.
- Ensure system security, scalability, and performance for future growth and feature expansion.

1.9 Main Features

Learnova provides a comprehensive set of intelligent features designed to improve the reading and knowledge discovery experience:

- AI-Powered Book Recommendation System.
- Semantic Search Engine.
- AI-Based Book Summarization.
- Direct Question & Answer from Book Content.
- Personal Library Management.
- Favorites and Bookmarking System.
- Note-Taking and Highlighting Features.
- Smart Notifications and Personalized Suggestions.
- Offline Access to Saved Books and Notes.

- User Authentication and Profile Management.
- Responsive Cross-Platform Mobile Application (Flutter).
- Administrative Dashboard for Content Management.

1.10 Scope and Limitations

1.10.1 Scope

The scope of Learnova focuses on developing an intelligent digital library platform that combines traditional library services with Artificial Intelligence technologies. The system includes:

- Digital book and resource management.
- AI-powered recommendation services.
- Semantic search functionality.
- Book summarization services.
- Question-answering from book content.
- Personal reading libraries.
- Bookmarks, notes, and favorites management.
- Smart notification services.
- Mobile application support for Android and iOS.
- Administrative management tools.

1.10.2 Limitations

Despite its capabilities, the system has several limitations:

- AI-generated summaries and answers depend on the quality and availability of book content.
- Recommendation accuracy improves over time as more user interaction data becomes available.
- Semantic search effectiveness depends on the quality of indexed resources and NLP models.
- Offline mode is limited to previously saved books and notes.
- The current version focuses on digital resources and does not manage physical library inventories.

- Features such as OCR and extended Audiobook capabilities are planned as future enhancements.

1.11 System Architecture Overview

Learnova follows a modular and scalable architecture consisting of four major layers:

Frontend Layer (Flutter/Dart) Provides a responsive and user-friendly mobile interface for Android and iOS devices. It handles user interaction, navigation, content presentation, and audio playback using BLoC state management.

Backend Layer (Laravel/PHP) Manages business logic, authentication via Laravel Sanctum, RESTful API services, and communication between system components.

Database Layer (MySQL) Stores user information, preferences, reading history, book metadata, and application data. Ensures efficient data retrieval and management.

AI Processing Layer (Python / LangChain) Implements recommendation systems, semantic search, book summarization, and intelligent question-answering functionalities using OpenAI and Gemini APIs via a RAG pipeline.

Together, these layers create a scalable and maintainable architecture that supports efficient communication between users, library resources, and AI services.

1.12 Methodology

Learnova follows the **Agile Software Development Methodology**. Agile promotes iterative development, continuous improvement, and effective collaboration among team members throughout the project lifecycle.

1.12.1 Main Phases

Phase 1: Planning and Analysis

- Requirements gathering and analysis.
- Identifying system objectives and user needs.
- Defining project scope and specifications.

Phase 2: System Design

- Designing system architecture.
- Creating database schemas and ER diagrams.
- Developing UI/UX wireframes using Figma.

Phase 3: Development

- Building the Flutter mobile application.
- Developing Laravel backend APIs and services.
- Implementing authentication and user management.

Phase 4: AI Integration

- Developing recommendation models.
- Implementing semantic search functionality.
- Integrating summarization and question-answering modules.

Phase 5: Testing

- Unit Testing.
- Integration Testing.
- System Testing.
- Performance and Security Testing.

Phase 6: Deployment and Maintenance

- Deploying backend services.
- Publishing mobile applications.
- Monitoring system performance and implementing future improvements.

1.12.2 Tools and Technologies**Table 1.1.** Tools and Technologies Used in Learnova

Category	Technologies
Frontend	Flutter (Dart), BLoC State Management
Backend	PHP Laravel, RESTful APIs, Laravel Sanctum
Database	MySQL
AI & ML	Python, LangChain, OpenAI API, Gemini API, RAG Pipeline
Dev & Deployment	Git & GitHub, Postman, Figma

1.13 Project Management

Effective project management practices are applied to ensure successful development, efficient collaboration, and timely project completion.

1.13.1 Timeline and Milestones

The project is divided into several major milestones:

- Requirements Analysis and Planning.
- System Design and Database Modeling.
- Mobile Application Development.
- Backend API Development.
- AI Module Development and Integration.
- System Integration and Testing.
- Documentation Preparation.
- Final Deployment and Presentation — *July 2026*.

1.13.2 Team Roles and Responsibilities

The project team consists of six members organized into three development groups:

Flutter Development Team (2 Members)

- Develop the mobile application UI and screens.
- Implement BLoC state management and navigation.
- Integrate frontend with backend APIs.

Backend Development Team (2 Members)

- Develop RESTful APIs using Laravel.
- Design and manage MySQL database structures.
- Implement authentication and authorization services.

AI Integration Team (2 Members)

- Develop the recommendation engine.
- Implement semantic search and RAG pipeline.
- Build book summarization and question-answering services.

Chapter Summary

This chapter introduced the Learnova project and discussed its background, target users, motivation, expected benefits, and problem statement. It also presented the project's objectives, contributions, main features, scope, system architecture, development methodology, and project management plan. By integrating Artificial Intelligence

technologies with digital library services, Learnova aims to transform traditional digital libraries into intelligent reading assistants that provide personalized recommendations, efficient content discovery, and enhanced user experiences. The next chapter reviews related work, existing systems, and technologies relevant to the proposed solution.

Chapter 2

Literature Review

Contents

2.1	Overview	13
2.2	Existing E-Learning and Audiobook Platforms	13
2.3	Comparison of Traditional and AI-Powered Platforms	14
2.4	Applications of AI in Education	14
2.5	Recommendation Systems in E-Learning	14
2.6	Retrieval-Augmented Generation (RAG) and LangChain	15
2.7	Text-to-Speech Technologies	16
2.8	Flutter as a Cross-Platform Framework	16
2.9	Related Works	16
2.10	Limitations of Existing Systems	17

2.1 Overview

This chapter presents a comprehensive review of the existing literature and related technologies relevant to the Learnova platform. The review covers existing e-learning and audiobook platforms, applications of Artificial Intelligence in education, recommendation systems, Retrieval-Augmented Generation (RAG) pipelines, Text-to-Speech technologies, and cross-platform mobile development using Flutter. The purpose of this review is to identify the limitations of current systems and justify the design decisions adopted in Learnova.

2.2 Existing E-Learning and Audiobook Platforms

The digital education and audiobook market has grown significantly over the past decade, giving rise to numerous platforms that serve learners worldwide. Understanding these platforms is essential to identifying gaps that Learnova aims to fill.

2.2.1 Popular Audiobook Platforms

Audible (Amazon) is the world's largest audiobook platform, offering over 500,000 titles with professional narration. While it provides a rich audio experience, it lacks intelligent content summarization, AI-powered Q&A, or personalized learning path features.

Spotify has expanded into audiobooks, integrating them alongside podcasts and music. Its recommendation engine is based on listening behavior, but it does not support educational features such as note-taking, bookmarks, or AI-generated summaries.

Google Play Books offers TTS functionality for e-books, allowing users to listen to any purchased book. However, the TTS quality relies on device-native engines and lacks contextual AI interaction with content.

Kobo provides e-reading and limited audiobook support, with basic annotation tools. It does not incorporate AI-driven recommendations or question-answering from book content.

2.2.2 Popular E-Learning Platforms

Coursera and **edX** offer structured online courses from universities worldwide. While they include video lectures and quizzes, they do not provide audiobook-style consumption or real-time AI interaction with course materials.

Khan Academy provides free educational content with adaptive exercises but does not support digital book management or AI-powered summarization.

2.3 Comparison of Traditional and AI-Powered Platforms

Traditional digital library and e-learning platforms primarily rely on keyword-based search and static content organization. Users are expected to manually browse large collections to locate relevant resources, which is time-consuming and often ineffective.

AI-powered platforms, by contrast, leverage machine learning models to understand user intent, personalize content delivery, and provide intelligent assistance. Table 2.1 summarizes the key differences between traditional and AI-powered platforms.

Table 2.1. Comparison of Traditional vs. AI-Powered E-Learning Platforms

Feature	Traditional Platforms	AI-Powered Platforms
Search	Keyword-based	Semantic / NLP-based
Recommendations	Manual / Category-based	Personalized ML models
Content Interaction	Read/Watch only	Q&A, Summarization, Chat
Audio Delivery	Limited / None	TTS with AI narration
User Adaptation	Static	Dynamic, behavior-driven
Personalization	Low	High

2.4 Applications of AI in Education

Artificial Intelligence has increasingly been applied in educational settings to enhance learning outcomes and improve user engagement. Tiwari [1] highlights that the integration of AI and machine learning in education has significant potential to personalize student learning experiences and improve academic performance.

Large Language Models (LLMs) such as GPT-4 and Gemini have opened new possibilities for educational tools. These models can generate explanations, answer questions, and summarize content with human-like fluency. Jayaram [2] discusses how AI-driven personalization can create hyper-personalized learning journeys tailored to each student type.

Natural Language Processing (NLP) techniques further enable systems to analyze the semantic meaning of user queries, going beyond simple keyword matching to understand context and intent.

2.5 Recommendation Systems in E-Learning

Recommendation systems are a critical component of modern e-learning platforms. da Silva et al. [3] conducted a systematic literature review on educational recommender systems and concluded that hybrid systems integrating collaborative filtering, content-

based approaches, and learner modeling demonstrate improved effectiveness in terms of learner engagement and outcomes.

A comprehensive review of recommender systems published in 2024 [4] synthesized recent advancements, including the increased use of deep learning architectures, graph-based models, and hybrid recommendation pipelines across domains including education.

Yanes et al. [5] proposed a machine learning-based recommender system for improving student learning experiences, achieving significant improvements in recommendation relevance through collaborative and content-based hybrid approaches.

In the context of Learnova, the recommendation engine analyzes user reading history, listening behavior, preferences, and ratings to suggest relevant books and educational materials.

2.6 Retrieval-Augmented Generation (RAG) and LangChain

Retrieval-Augmented Generation (RAG) is an AI framework that combines the generative capabilities of large language models with information retrieval from external knowledge bases. Instead of relying solely on pre-trained knowledge, RAG systems retrieve relevant documents at inference time and use them to ground the model's responses.

EL Maazouzi et al. [6] demonstrated the effectiveness of synergistically integrating LangChain, GPT models, and RAG for optimizing recommendation systems in e-learning environments. Their work confirms that RAG pipelines significantly improve the relevance and accuracy of AI-generated responses in educational contexts.

LangChain, released in 2022, is a versatile framework designed for building applications that leverage large language models [7]. It simplifies the integration of LLMs with external data sources, enabling developers to create RAG systems that combine generative models with retrieval systems. LangChain provides a robust API for connecting LLMs to vector databases, document stores, and external APIs.

Burgan et al. [8] developed a RAG chatbot application using LangChain and adaptive LLMs to assist university students in navigating institutional information, demonstrating the practical feasibility of LangChain-based RAG systems in higher education settings.

In Learnova, the RAG pipeline processes uploaded PDF books, chunks them into searchable segments, stores them in a vector database, and retrieves relevant passages to answer user questions with contextual accuracy.

2.7 Text-to-Speech Technologies

Text-to-Speech (TTS) technology converts written text into spoken audio, enabling audio-first consumption of digital content. Modern TTS systems powered by deep learning produce natural-sounding speech that closely resembles human narration.

In the context of mobile applications, TTS has become increasingly accessible through APIs and cross-platform frameworks. The `flutter_tts` package provides a Flutter plugin that leverages native TTS engines on Android (`TextToSpeech`) and iOS (`AVSpeechSynthesizer`), enabling multilingual speech synthesis with configurable parameters such as pitch, rate, and volume [9].

Commercial TTS APIs such as Google Text-to-Speech, Amazon Polly, and ElevenLabs offer high-quality neural voices suitable for audiobook-style delivery. These APIs support multiple languages, voice styles, and SSML (Speech Synthesis Markup Language) for fine-grained control over pronunciation and intonation.

Learnova integrates a TTS pipeline that converts PDF book content into high-quality audio, delivering a Spotify-inspired listening experience with playback speed control, chapter navigation, and a persistent Mini Player.

2.8 Flutter as a Cross-Platform Framework

Flutter is an open-source UI framework developed by Google that enables the creation of natively compiled applications for Android, iOS, web, and desktop from a single codebase using the Dart programming language.

Flutter’s widget-based architecture and the BLoC (Business Logic Component) state management pattern provide a clean separation between UI and business logic, making it well-suited for complex, data-driven applications such as Learnova. The BLoC pattern ensures predictable state transitions and facilitates testability.

The availability of packages such as `just_audio` for audio playback, `flutter_tts` for TTS, and `dio` for HTTP communication makes Flutter particularly effective for building feature-rich audiobook and e-learning applications.

2.9 Related Works

Several prior works are directly relevant to the design and implementation of Learnova:

RamChat [8]: A RAG-based chatbot developed at Shepherd University using LangChain and LLMs to assist students with institutional information. This work validates the use of LangChain-based RAG pipelines in educational environments, a core component

of Learnova's Q&A feature.

Personalized E-Learning Recommender System Based on Autoencoders

[10]: This study proposed using deep autoencoder networks to address the sparsity problem in collaborative filtering for e-learning recommendation, achieving improved personalization performance.

A Digital Recommendation System for Personalized Learning [11]: A review published in IEEE examining 60 articles on e-learning recommendation systems, identifying collaborative and content-based approaches as dominant methods with a recent shift toward machine learning.

AI-Driven Solutions for Library User Experiences [12]: This work explored how AI can improve library services through personalized knowledge dissemination and inclusive user experiences, directly informing Learnova's intelligent library management approach.

2.10 Limitations of Existing Systems

Based on the literature review, the following limitations are identified in existing systems:

- **Lack of audio-first learning:** Most digital library and e-learning platforms do not provide high-quality, AI-narrated audiobook experiences integrated with learning tools.
- **Limited AI interaction:** Existing audiobook platforms such as Audible and Spotify do not allow users to interact with book content through Q&A or summarization.
- **Weak personalization:** Traditional platforms rely on category-based browsing rather than behavior-driven recommendation engines tailored to individual learning patterns.
- **No RAG-based Q&A from books:** Current systems do not support contextual question-answering directly from uploaded educational materials using RAG pipelines.
- **Fragmented experience:** Audio consumption, reading, annotation, and AI assistance are spread across multiple disconnected applications, creating friction in the learning workflow.

Learnova addresses these limitations by unifying audio-first consumption, AI-powered interaction, personalized recommendations, and learning tools within a single, coherent mobile platform.

Chapter Summary

This chapter reviewed the existing literature on e-learning and audiobook platforms, AI applications in education, recommendation systems, RAG pipelines, TTS technologies, and cross-platform development with Flutter. The review identified key limitations in current systems, including the absence of integrated audio-first learning, limited AI interaction with book content, and weak personalization capabilities. These findings provide the foundation for the design decisions adopted in Learnova, which is presented in the following chapters.

Chapter 3

Backend Implementation

Contents

4.1 Overview 20

4.2 Objectives 20

4.3 Scope 21

4.4 System Architecture 21

4.5 Database Design 23

4.6 API Documentation 25

4.7 AI Microservices 28

4.8 Admin Panel (FilamentPHP 3) 30

4.9 Testing 31

4.10 Challenges Faced 32

4.11 Future Work 33

3.1 Overview

The modern reader, student, and researcher face a growing challenge: the sheer volume of available books and digital resources makes it increasingly difficult to discover content that truly matches one's interests, academic needs, or research goals. Traditional library systems rely on keyword-based search, which often returns irrelevant results and fails to understand the user's intent. There is also a lack of platforms that combine book management, educational courses, intelligent recommendations, and AI-powered features in one unified system.

Learnova addresses these challenges by building a centralized, scalable, and AI-integrated backend that powers personalized book and course discovery, semantic search, book summarization, and direct Q&A from book content — all through a clean RESTful API consumed by the Flutter mobile application.

3.2 Objectives

The main objectives of the backend system are:

- To provide a secure RESTful API consumed by the Flutter mobile application.
- To implement full CRUD operations for books, courses, authors, categories, and users.
- To integrate JWT-based authentication and role-based access control (Admin, Researcher, Student).
- To support a review and rating system allowing users to evaluate books.
- To manage user bookmark collections, personal reading shelves, and favorites.
- To integrate 6 Python AI microservices covering ingestion, recommendations, semantic search, summarization, and Q&A.
- To support course management with video lessons and AI-powered lesson transcription.
- To deliver an admin panel (FilamentPHP 3) for efficient management of all platform resources.
- To implement OTP-based password reset and social login (Google/Facebook).

3.3 Scope

3.3.1 Included

- User registration, login, OTP password reset, and social login (Google/Facebook).
- JWT-based authentication via Laravel Sanctum with role-based access control (admin, researcher, student).
- Full CRUD for Books, Courses, Course Videos, Authors, Categories, and Chapters.
- Advanced search and filtering: global search, semantic search, search history, and trending.
- Review and rating system for books.
- Bookmark, favorites, and personal library shelves management.
- Onboarding: user topic preferences and profile setup.
- Admin panel (FilamentPHP 3) with full CRUD, file uploads, and analytics dashboard.
- Full AI integration: 6 Python microservices via Docker (ingestion, embedding, recommendations, semantic search, summarization, RAG Q&A).
- AI-powered lesson transcription via `TranscribeAllLessons` command.
- Proxy architecture: Laravel acts as a gateway forwarding requests to AI microservices.

3.3.2 Excluded

- Video conferencing or live streaming functionality.
- Payment processing and subscription management.
- Direct instructor consultation chat.
- OCR for scanned documents (planned future enhancement).
- Audio-book streaming (planned future enhancement).

3.4 System Architecture

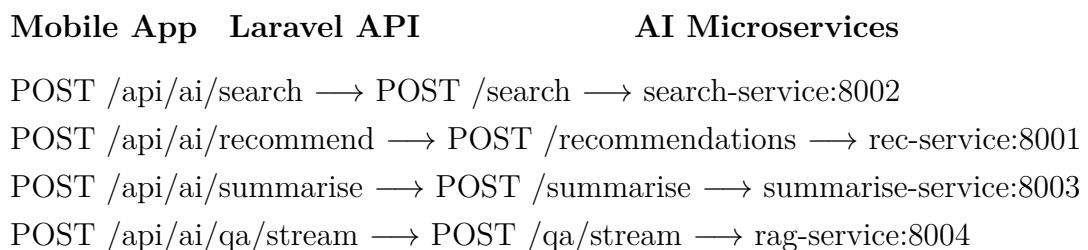
The Learnova backend follows a layered RESTful API architecture built on **Laravel 12**. The system is designed around a proxy pattern where Laravel serves as the central gateway: it handles authentication, business logic, and database operations, while forwarding AI-related requests to dedicated Python microservices running in Docker containers.

Table 3.1. System Architecture Layers and Responsibilities

Layer	Technology	Responsibility
Mobile Frontend	Flutter (Dart)	Cross-platform UI, state management, API consumption
Backend API	Laravel 12 (PHP 8.1+)	RESTful endpoints, business logic, authentication, AI proxy
Admin Panel	FilamentPHP 3	Admin UI, CRUD management, analytics dashboard
Database	MySQL 8.0	Persistent storage for users, books, courses, reviews
AI — Ingestion	Python (Port 8000)	PDF/EPUB upload, text extraction, chunking, Kafka
AI — Embedding	Python (Background)	Kafka consumer, vector generation, Pinecone storage
AI — Recommendations	Python (Port 8001)	NCF + CBF hybrid recommendation engine
AI — Semantic Search	Python (Port 8002)	Semantic search and result re-ranking
AI — Summarization	Python (Port 8003)	FLAN-T5 book summarization
AI — RAG Q&A	Python (Port 8004)	Groq LLM Q&A with citations (sync + SSE streaming)

3.4.1 AI Proxy Architecture

All AI-related API calls follow a proxy pattern. The mobile app sends requests to Laravel, which forwards them to the appropriate AI microservice and returns the result. If an AI service is unavailable, Laravel returns a graceful error message rather than crashing the application. Figure 3.1 illustrates this flow.

**Figure 3.1.** AI Proxy Request Flow in Learnova

3.5 Database Design

The system uses **MySQL 8.0** as the primary relational database. The schema is normalized with clear foreign-key relationships between all entities. Laravel 12 migrations are used to version-control the schema. The database covers user management, book and course content, AI metadata, community features, and personal library management.

3.5.1 users

Table 3.2. users Table Schema

Column	Type	Description
id	BIGINT (PK)	Auto-incremented primary key
name	VARCHAR(255)	Full name of the user
email	VARCHAR(255)	Unique email address
password	VARCHAR(255)	Bcrypt-hashed password
role	ENUM	Role: admin, researcher, student
email_verified_at	TIMESTAMP	Email verification timestamp
created_at / updated_at	TIMESTAMP	Record timestamps

3.5.2 books

Table 3.3. books Table Schema

Column	Type	Description
id	BIGINT (PK)	Auto-incremented primary key
title	VARCHAR(255)	Book title
isbn	VARCHAR(255)	Unique ISBN (used as AI service key)
author_id	BIGINT (FK)	Reference to authors table
category_id	BIGINT (FK)	Reference to categories table
description	TEXT	Book synopsis
cover_image	VARCHAR(255)	Path to uploaded cover image
file_path	VARCHAR(255)	Path to PDF/EPUB file for AI ingestion
published_year	YEAR	Year of publication
is_free	BOOLEAN	Whether the book is free or paid
views_count	INT	View counter for trending analytics
created_at / updated_at	TIMESTAMP	Record timestamps

3.5.3 courses

Table 3.4. courses Table Schema

Column	Type	Description
id	BIGINT (PK)	Auto-incremented primary key
title	VARCHAR(255)	Course title
description	TEXT	Course description
category_id	BIGINT (FK)	Reference to categories table
cover_image	VARCHAR(255)	Course thumbnail image
is_free	BOOLEAN	Whether the course is free or paid
is_featured	BOOLEAN	Flag for featured courses
created_at / updated_at	TIMESTAMP	Record timestamps

3.5.4 course_videos

Table 3.5. course_videos Table Schema

Column	Type	Description
id	BIGINT (PK)	Auto-incremented primary key
course_id	BIGINT (FK)	Reference to courses table
title	VARCHAR(255)	Video lesson title
video_url	VARCHAR(255)	URL or path to video file
transcript	TEXT	AI-generated transcript (TranscribeAllLessons)
duration	INT	Video duration in seconds
order	INT	Display order within the course
created_at / updated_at	TIMESTAMP	Record timestamps

3.5.5 reviews

Table 3.6. reviews Table Schema

Column	Type	Description
id	BIGINT (PK)	Auto-incremented primary key
user_id	BIGINT (FK)	Reference to users table
book_id	BIGINT (FK)	Reference to books table
rating	TINYINT (1–5)	Star rating
comment	TEXT	Optional review text
created_at / updated_at	TIMESTAMP	Record timestamps

3.5.6 bookmarks and personal_shelves

Table 3.7. bookmarks / personal_shelves Table Schema

Column	Type	Description
id	BIGINT (PK)	Auto-incremented primary key
user_id	BIGINT (FK)	Reference to users table
book_id	BIGINT (FK)	Reference to books table
shelf_type	ENUM	Type: reading, completed, favorite
created_at / updated_at	TIMESTAMP	Record timestamps

3.5.7 ai_metadata

Table 3.8. ai_metadata Table Schema

Column	Type	Description
id	BIGINT (PK)	Auto-incremented primary key
book_id	BIGINT (FK)	Reference to books table
summary	TEXT	AI-generated summary (FLAN-T5)
keywords	JSON	Extracted keywords array
vector_status	ENUM	Embedding status: pending, processing, done
mindmap_data	JSON	Optional visual mind map data
created_at / updated_at	TIMESTAMP	Record timestamps

3.6 API Documentation

The backend exposes a comprehensive RESTful API. All endpoints return JSON responses. Authentication is handled via Laravel Sanctum Bearer tokens passed in the `Authorization` header. The base URL for all API calls is `http://localhost:8000/api`.

3.6.1 Authentication Endpoints

Table 3.9. Authentication API Endpoints

Method	Endpoint	Description
POST	/api/register	Register a new user account
POST	/api/login	Login and receive Sanctum token
POST	/api/logout	Invalidate current token
POST	/api/forgot-password/send-otp	Send OTP for password reset
POST	/api/forgot-password/verify-otp	Verify the received OTP code
POST	/api/forgot-password/reset	Reset password after OTP verification
GET	/api/auth/{provider}/redirect	Redirect to Google or Facebook OAuth
GET	/api/auth/{provider}/callback	OAuth callback handler (social login)

3.6.2 Books and Discovery Endpoints

Table 3.10. Books and Discovery API Endpoints

Method	Endpoint	Description
GET	/api/home	Home page: personalized recommendations
GET	/api/discover	Discover page: trending and featured content
GET	/api/discover/trending-20	Top 20 trending books by views count
GET	/api/discover/free-20	Top 20 free books
GET	/api/search	General search across books and content
GET	/api/search/global	Global comprehensive search
GET	/api/search/filter/{type}	Filtered search by type (book/author/-course)
DELETE	/api/search/history/{id?}	Delete search history entry
POST	/api/books/{id}/favorite	Toggle book favorite status
POST	/api/books/{id}/listen	Record a book listen/view event
GET	/api/books/{id}/share	Get shareable link for a book

3.6.3 Courses Endpoints

Table 3.11. Courses API Endpoints

Method	Endpoint	Description
GET	/api/courses	List all courses with pagination
GET	/api/courses/featured	Get featured courses
GET	/api/courses/free	Get free courses
GET	/api/courses/category/{id}	Filter courses by category
GET	/api/courses/{id}	Get course details including video list
POST	/api/courses	Create a new course (Admin only)
PUT	/api/courses/{id}	Update course details (Admin only)
DELETE	/api/courses/{id}	Delete a course (Admin only)
GET	/api/courses/{id}/videos	Get all videos for a course
POST	/api/courses/{id}/videos	Add a new video to a course
PUT	/api/courses/{id}/videos/{vidId}	Update a course video
DELETE	/api/courses/{id}/videos/{vidId}	Delete a course video

3.6.4 AI Services Endpoints

All AI endpoints follow a proxy pattern: Laravel receives the request, validates authentication, and forwards to the appropriate Python microservice. If a service is unavailable, a graceful error is returned.

Table 3.12. AI Services API Endpoints

Method	Endpoint	Service	Description
GET	/api/ai/health	All	Health check for all AI microservices
POST	/api/ai/ingest	:8000	Upload PDF/EPUB for processing
GET	/api/ai/ingest/{isbn}/status	:8000	Track ingestion pipeline progress
POST	/api/ai/recommendations	:8001	Get personalized book recommendations
GET	/api/ai/recommendations/cold-start	:8001	Cold-start recommendations
POST	/api/ai/search/semantic	:8002	Semantic search by query
POST	/api/ai/search/similar/{isbn}	:8002	Find similar books by ISBN
POST	/api/ai/summarise	:8003	Summarize a book by ISBN
GET	/api/ai/summarise/{isbn}/progress	:8003	Track summarization progress
POST	/api/ai/qa/sync	:8004	Full Q&A response with citations
POST	/api/ai/qa/stream	:8004	Streaming Q&A via SSE tokens

3.7 AI Microservices

The AI layer consists of 6 Python microservices deployed via Docker Compose. Each service is independently scalable and communicates through well-defined REST interfaces. The services use **Kafka** for asynchronous pipeline processing and **Pinecone** as the vector database for semantic search capabilities.

Table 3.13. Python AI Microservices Overview

Service	Port	Model/Tech	Function
ingestion	8000	PyMuPDF + Kafka	PDF/EPUB upload, text extraction, chunking
embedding	—	Kafka + Pinecone	Background: generate vectors, store in Pinecone
rec_service	8001	NCF + CBF hybrid	Personalized recommendations
search_service	8002	Hugging Face + re-ranker	Semantic search with intent understanding
summarise_service	8003	FLAN-T5	Multi-type summarization: short, detailed, mind map
rag_service	8004	Groq LLM	Q&A from book text with citations (sync + SSE)

3.7.1 Courses AI — TranscribeAllLessons

As part of the courses AI integration, a dedicated Laravel Artisan command **TranscribeAllLessons** was developed. This command processes course video lessons by sending them to the AI transcription pipeline, generating text transcripts stored in the `course_videos` table. This enables searchable lesson content, AI-powered lesson summaries, and accessibility improvements for hearing-impaired users.

Listing 3.1. Running the TranscribeAllLessons command

```

1 # Run transcription for all unprocessed lessons
2 php artisan transcribe:all-lessons
3
4 # The command:
5 # 1. Fetches all course_videos without transcripts
6 # 2. Sends each video to the AI transcription service
7 # 3. Stores the returned transcript in the database
8 # 4. Logs progress and handles failures gracefully

```

3.7.2 AI Service Configuration

All AI service URLs and timeout settings are centralized in `config/ai.php`. The `AiService.php` class in `app/Services` provides a unified interface for all AI microservice calls, handling connection errors gracefully.

Table 3.14. AI Service Configuration Parameters

Config Key	Default Value	Description
ai.ingestion_url	http://localhost:8000	Ingestion service base URL
ai.rec_url	http://localhost:8001	Recommendation service base URL
ai.search_url	http://localhost:8002	Semantic search service base URL
ai.summarise_url	http://localhost:8003	Summarization service base URL
ai.rag_url	http://localhost:8004	RAG Q&A service base URL
ai.timeout	30 seconds	Default HTTP timeout for AI requests

3.8 Admin Panel (FilamentPHP 3)

The backend includes a full-featured admin panel built with **FilamentPHP 3**, accessible at `/admin`. The panel is organized into four sections covering the entire platform.

Table 3.15. Admin Panel Sections and Resources

Section	Resources	Description
Library	Books, Courses, Authors, Categories, Chapters	Full CRUD with file uploads
Community	Reviews, Personal Shelves	User-generated content moderation
AI Engine	AI Metadata	View summaries, keywords, vector status
Administration	Users	User management with role assignment

Table 3.16. Admin Panel Default Credentials

Field	Value
Admin Panel URL	http://127.0.0.1:8000/admin
Default Email	admin@admin.com
Default Password	password
Note	Change credentials immediately after first deployment

3.9 Testing

The backend APIs were tested using **Postman** for manual endpoint validation and Laravel’s built-in **PHPUnit** framework for automated tests. Each endpoint was verified for correct HTTP response codes, data accuracy, authentication enforcement, and error handling for invalid inputs.

3.9.1 Testing Tools

Table 3.17. Testing Tools and Scope

Tool	Scope
Postman	Manual API testing: auth flows, CRUD operations, AI endpoints, edge cases
PHPUnit	Automated unit and integration tests for models and API responses
Browser (Manual)	Admin panel CRUD, file uploads, dashboard analytics
Docker Health Checks	Verify all 6 AI microservices via <code>/health</code> endpoints

3.9.2 Sample Test Cases

Table 3.18. Sample API Test Cases and Expected Results

Endpoint	Method	Test Case	Expected Result
/api/register	POST	Valid data	201 Created + User object
/api/register	POST	Duplicate email	422 Unprocessable Entity
/api/login	POST	Valid credentials	200 OK + Sanctum token
/api/login	POST	Wrong password	401 Unauthorized
/api/forgot-password/send-otp	POST	Valid phone	200 OK + OTP sent
/api/discover/trending-20	GET	Fetch trending	200 OK + 20 books
/api/courses	GET	List all courses	200 OK + Paginated
/api/ai/health	GET	All services running	200 OK + All healthy
/api/ai/search/semantic	POST	Valid query	200 OK + Ranked results
/api/ai/recommendations	POST	Valid user_id	200 OK + Recommended books
/api/ai/summarise	POST	Valid ISBN + type	200 OK + Summary text
/api/ai/qa/sync	POST	Question about book	200 OK + Answer + citations
/api/bookmarks	GET	No auth token	401 Unauthorized

3.10 Challenges Faced

- **AI Microservices Integration:** Designing a proxy architecture in Laravel that gracefully handles failures from any of the 6 Python microservices without crashing

the main application required careful error handling and timeout configuration in `AIService.php`.

- **Docker Environment Configuration:** Setting up Kafka, Pinecone, and 6 containerized services with correct environment variables and ensuring inter-container networking required extensive troubleshooting of the `docker-compose.yml` configuration.
- **Courses AI — TranscribeAllLessons:** Developing the Artisan command to batch-process course video lessons through the AI transcription pipeline required handling partial failures, retry logic, and efficient progress logging for large course libraries.
- **Dual Authentication System:** Configuring Laravel Sanctum for API JWT tokens alongside FilamentPHP's session-based admin authentication without conflicts required careful guard configuration in `auth.php`.
- **Social Login Integration:** Implementing Google and Facebook OAuth flows with proper callback handling, user creation/linking logic, and token generation for the mobile app required careful configuration of Laravel Socialite.
- **Role-Based Access Control:** Implementing middleware that correctly restricts admin-only endpoints while keeping public discovery endpoints open required thorough testing across three user roles.
- **Database Migration Ordering:** Managing foreign-key dependencies across multiple related tables required careful ordering of Laravel migrations to avoid constraint violations.

3.11 Future Work

- Merge the courses AI branch (`TranscribeAllLessons`) into main and extend to support real-time transcription during video upload.
- Add audio-book streaming with AI-generated narration using text-to-speech models.
- Implement OCR for scanned document digitization using Tesseract or similar tools.
- Develop an advanced admin analytics dashboard with user engagement metrics, book popularity trends, and AI usage statistics.
- Expand the recommendation engine to include course recommendations alongside book recommendations.

- Add multi-language support (Arabic/English) across all API responses and the admin panel.
- Implement offline mode with client-side caching for saved books and notes.
- Add direct Q&A for course lesson transcripts using the existing RAG infrastructure.

Chapter Summary

This chapter documented the backend implementation of Learnova — a robust, secure, and AI-integrated system built on Laravel 12, FilamentPHP 3, MySQL 8.0, and a suite of 6 Python AI microservices. The proxy architecture ensures that the platform degrades gracefully when AI services are unavailable, maintaining core functionality at all times. The **TranscribeAllLessons** command demonstrates the system's extensibility, enabling AI features to be added incrementally to existing content modules. The admin panel provides administrators with a powerful graphical interface for managing all platform content. The next chapter details the intelligent AI features built on top of this backend infrastructure.

Chapter 4

Intelligent Features of the System

Contents

5.1	Overview	36
5.2	Ingestion Service — Document Processing Pipeline	36
5.3	Embedding Service — Semantic Vectorisation	37
5.4	Search Service — Two-Stage Semantic Retrieval	39
5.5	Summarisation Service	41
5.6	RAG Q&A Service	42
5.7	Recommendation Service — Hybrid NCF + CBF	44
5.8	Ethical Considerations	46

4.1 Overview

This chapter provides a comprehensive technical description of the six AI subsystems embedded in the Learnova platform: the Ingestion Service, the Embedding Service, the Search Service, the Summarisation Service, the Retrieval-Augmented Generation (RAG) Q&A Service, and the Recommendation Service. Together, these services transform a static collection of book files into a dynamic, queryable knowledge base that users can explore through natural language, receive personalised recommendations, and query for specific information — without any manual indexing or metadata entry.

4.2 Ingestion Service — Document Processing Pipeline

The Ingestion Service is the entry point for all book content in the platform. It accepts PDF and ePub files uploaded by library administrators and orchestrates a three-stage pipeline: text extraction, cleaning and sentence tokenisation, and sliding-window chunking. The output is a stream of text chunks published to Kafka, which the Embedding Service then consumes asynchronously to produce vector representations.

4.2.1 Text Extraction

Format detection is based on file extension. PDF files are processed using **PyMuPDF** (**fitz**), which extracts plain text page by page. ePub files are processed using **ebooklib**, which iterates over HTML chapter items, strips all HTML tags, and yields plain text sections. In both cases, pages or sections with fewer than 50 characters are silently skipped, filtering out blank pages, cover images, and decorative separators that would otherwise produce noise in the vector index.

4.2.2 Cleaning and Chunking

Extracted text is cleaned and sentence-tokenised before chunking. The chunker uses the **all-mpnet-base-v2** tokenizer to measure token counts precisely, ensuring that chunks match the embedding model’s input space. A sliding-window algorithm fills each chunk greedily up to 512 tokens, then carries the last 50 tokens into the next chunk as overlap — preserving cross-boundary semantic continuity. Sentences that individually exceed 512 tokens are split into sub-windows before chunking begins.

Each chunk is assigned a deterministic identifier of the form `{isbn}_p{page_num}_c{chunk_index}`, enabling exact traceability from any retrieved vector back to its source page in the original book.

Table 4.1. Ingestion Service Chunking Parameters

Parameter	Value	Config Key	Effect
Chunk Size	512 tokens	CHUNK_SIZE	Maximum tokens per chunk
Chunk Overlap	50 tokens	CHUNK_OVERLAP	Tokens carried into next chunk
Min Page Length	50 chars	(hardcoded)	Filters blank/decorative pages
Upsert Batch Size	100 vectors	UPSERT_BATCH_SIZE	Vectors flushed per Pinecone call

4.2.3 Kafka Publishing and Status Tracking

Each chunk is serialised as a JSON message and published to the Kafka topic `book.chunks` with the book’s ISBN as the partition key, ensuring all chunks from the same book are processed in order by the same Embedding Service consumer instance. Ingestion runs as a FastAPI background task: the API accepts the upload, stores the file to disk, and immediately returns HTTP 202 Accepted with a job ID. The caller can poll `/ingest/{isbn}/status` to track progress through the `queued` → `processing` → `done` lifecycle, with status persisted in Redis.

4.3 Embedding Service — Semantic Vectorisation

The Embedding Service is the foundational layer of the platform’s AI pipeline. It converts raw text chunks, produced by the Ingestion Service, into dense 768-dimensional numerical vectors that encode semantic meaning. These vectors are stored in Pinecone and serve as the retrieval backbone for every downstream AI capability: semantic search, recommendations, summarisation, and RAG Q&A.

4.3.1 Design Rationale

Traditional library search systems rely on keyword matching over metadata fields such as title, author, and ISBN. This approach fails to surface semantically relevant content buried within the book text itself. Semantic search addresses this limitation by representing every passage as a point in a shared vector space: two passages with similar meaning are geometrically close, even if they share no words in common.

The Embedding Service is designed as a headless Kafka consumer with no HTTP interface of its own. This architectural choice decouples embedding from the upload API, allowing large books to be indexed asynchronously without blocking the user-facing

request. Multiple consumer instances can be deployed independently to scale ingestion throughput.

4.3.2 Model: all-mpnet-base-v2

The `sentence-transformers/all-mpnet-base-v2` model was selected as the embedding backbone after evaluation on the Semantic Textual Similarity (STS) benchmark. It is a fine-tuned variant of Microsoft’s MPNet architecture, trained on over one billion sentence pairs, producing 768-dimensional vectors with state-of-the-art performance on passage-level retrieval tasks.

A critical mathematical property is L2 normalisation: all output vectors are projected onto the unit hypersphere. Under this normalisation, cosine similarity equals the dot product of two vectors — a property exploited by Pinecone’s ANN index through maximum inner-product search, enabling sub-millisecond retrieval across millions of vectors.

Table 4.2. Embedding Model Configuration and Design Decisions

Property	Value	Justification
Model	all-mpnet-base-v2	Top STS benchmark performance on passage retrieval
Output Dimension	768-d float32	Balance between representational power and storage cost
Normalisation	L2 (unit sphere)	Cosine similarity = dot product, exploited by Pinecone ANN
Batch Size	64 texts / forward pass	Optimal GPU utilisation without out-of-memory risk
Hardware	CUDA GPU (CPU fallback)	Automatic device selection at runtime
Lazy Loading	On first <code>encode()</code> call	Avoids GPU initialisation overhead at service startup

4.3.3 BookEncoder Implementation

The `BookEncoder` class wraps the `SentenceTransformer` library and exposes two methods: `encode_single()` for real-time query encoding and `encode_batch()` for bulk ingestion. The `torch.no_grad()` context manager disables gradient computation during inference, reducing memory usage and improving throughput by approximately 30%.

Listing 4.1. `BookEncoder` `encode_batch` method


```

1 def encode_batch(self, texts: list[str], batch_size: int = 64) ->
  np.ndarray:
2     model = self._ensure_model()           # lazy-load on first call
3     with torch.no_grad():                  # disable gradients for
        inference
4         vecs = model.encode(
5             texts, batch_size=batch_size,
6             convert_to_numpy=True,
7             normalize_embeddings=True, # apply L2 normalisation
8         )
9     return vecs.astype(np.float32)         # 768-d float32 matrix

```

4.3.4 Kafka-to-Pinecone Pipeline and Fault Tolerance

The `EmbeddingBatchJob` class buffers up to 100 vector records in memory and flushes them to Pinecone as a single batched upsert, reducing API call overhead. Each vector is stored in a Pinecone namespace equal to the book’s ISBN, enabling efficient per-book filtering without maintaining separate indexes.

Upsert operations are retried up to three times with exponential back-off (delays of 1s, 2s, and 4s). If all three attempts fail, the batch is forwarded to a Kafka dead-letter topic (`book.chunks.dlq`) for manual inspection and replay.

Table 4.3. Embedding Batch Job Processing Stages

Stage	Input	Output	Technology
Consume	Kafka: book.chunks	ChunkData object	aiokafka Kafka-Consumer
Encode	Chunk text (string)	768-d float32 vector	SentenceTransformer
Buffer	Vector record + metadata	In-memory list (max 100)	Python list
Upsert	List of records	Pinecone index entry	Pinecone client upsert()
DLQ	Failed batch	Kafka: book.chunks.dlq	aiokafka KafkaProducer

4.4 Search Service — Two-Stage Semantic Retrieval

The Search Service provides natural-language book search across the entire library catalogue. Unlike traditional keyword search, the service encodes the query into a 768-dimensional vector and retrieves semantically similar passages — surfacing relevant content even when the query shares no exact words with the text. To balance recall

and precision, the service implements a two-stage retrieval pipeline combining a fast bi-encoder for candidate recall with a slower but more accurate cross-encoder for re-ranking.

4.4.1 Two-Stage Pipeline

Stage 1 — Approximate Nearest Neighbour (ANN) Retrieval: The user’s query is encoded by the same `all-mpnet-base-v2` bi-encoder used during ingestion, producing an L2-normalised 768-dimensional vector. This vector is submitted to Pinecone’s global namespace, retrieving the top 50 candidates (`ANN_TOP_K = 50`) by cosine similarity. An optional genre filter can be applied as a Pinecone metadata filter to restrict results to specific categories.

Stage 2 — Cross-Encoder Re-ranking: The 50 ANN candidates are passed to a CrossEncoder model (`cross-encoder/ms-marco-MiniLM-L-6-v2`), which scores each query–passage pair jointly. Unlike the bi-encoder, which encodes query and passage independently, the cross-encoder attends to both simultaneously — producing more accurate relevance scores at the cost of higher latency. The top 10 results (`FINAL_TOP_K = 10`) by cross-encoder score are returned to the caller.

Table 4.4. Search Service Two-Stage Pipeline

Stage	Model	Output
ANN Retrieval	<code>all-mpnet-base-v2</code> (bi-encoder)	Top 50 candidates from Pinecone global namespace
Re-ranking	<code>cross-encoder/ms-marco-MiniLM-L-6-v2</code>	Top 10 results sorted by cross-encoder score
Similar Books	CBF (Pinecone synopsis namespace)	Books semantically similar to a given ISBN

4.4.2 Caching and Similar-Book Search

Search results are cached in Redis for 60 seconds using a SHA-256 hash of the query, top-k, and genre filter as the cache key. This avoids redundant Pinecone queries for repeated searches within the same minute. The service also exposes a `/search/similar/{isbn}` endpoint that delegates to the Content-Based Filtering layer of the Recommendation Service, returning books whose synopsis vectors are closest to the requested ISBN in Pinecone’s synopsis namespace.

4.5 Summarisation Service

The Summarisation Service generates concise textual summaries of full book content and structured mind-map representations of key themes. Because a full book may contain thousands of text chunks — far exceeding any language model context window — the service employs a map-reduce strategy to handle arbitrarily long documents.

4.5.1 Model: FLAN-T5-large

The service uses Google’s **FLAN-T5-large**, a sequence-to-sequence model based on the T5 architecture. FLAN-T5 was instruction-fine-tuned on a large collection of tasks expressed as natural language instructions, enabling it to follow structured prompts reliably without task-specific fine-tuning — making it well-suited for a library system where content varies widely in genre and domain.

Beam search (`num_beams=4`) explores multiple candidate sequences simultaneously, consistently producing higher-quality summaries than greedy decoding. A `no_repeat_ngram_size` of 3 suppresses repetitive phrase patterns, a common failure mode in extractive-style models.

Table 4.5. FLAN-T5 Prompt Configurations by Summary Type

Type	Prompt Role	Max Tokens	Use Case
SHORT	Book blurb writer — 2–3 engaging sentences for a general reader	150	Quick overview
DETAILED	Literary analyst — thematic summary with key events and characters	300	Deep-dive analysis

4.5.2 Map-Reduce Pipeline

The `MapReducePipeline` processes books in two phases to overcome the context window constraint:

MAP Phase: Chunks are fetched from Pinecone in page-number order and divided into batches of 5 consecutive chunks. Each batch is independently summarised by FLAN-T5 with an 80-token output cap, producing a list of intermediate summaries. Batches are processed sequentially to respect GPU memory constraints.

REDUCE Phase: All intermediate summaries are concatenated into a single string and passed through FLAN-T5 one final time with the user-selected type — `SHORT` (150 tokens) or `DETAILED` (300 tokens) — producing the final book summary.

The final summary is cached in Redis with a TTL of 24 hours, so repeated requests

for the same book are served instantly without re-running the pipeline. A GPU queue counter in Redis limits concurrent summarisation jobs to a maximum of 4. If the queue is full, the API returns HTTP 503 with a **Retry-After** header.

4.5.3 Mind Map Generation

In addition to textual summaries, the service can generate a structured JSON mind-map tree representing the book’s key themes. The **MindMapBuilder** class prompts FLAN-T5 to produce a JSON object with a root node (the book title) and child nodes (main themes), each containing a list of supporting subpoints. All output is validated against a Pydantic schema (**MindMapOutput**). If validation fails, a rule-based fallback generator guarantees a valid, well-structured response.

Table 4.6. Mind Map Generation Pipeline with Fallback Strategy

Stage	Input	Output	Fallback?
Prompt	Book summary (max 3000 chars)	Structured prompt	No
Generation	Prompt + FLAN-T5	Raw text output	No
JSON Extraction	Raw text	Parsed dict object	Yes — rule-based
Pydantic Validation	dict object	MindMapOutput	Yes — rule-based
Response	MindMapOutput	JSON tree	Fallback always valid

4.6 RAG Q&A Service

The RAG Service enables users to pose natural-language questions about any book in the library and receive accurate, page-cited answers. Rather than relying on a language model’s parametric knowledge, the service first retrieves the most relevant passages from the book and provides them as explicit context to the LLM, grounding its response in the actual text.

4.6.1 Architecture

The service is built on the **LangChain** framework with **Groq’s Llama-3.3-70b-Versatile** as the generation model and Pinecone as the vector retrieval backend. The pipeline consists of three sequential components: a retriever, a chain, and a streaming layer.

Table 4.7. RAG Service Component Overview

Component	Technology	Responsibility
Retriever	PineconeBookRetriever	Encodes query, fetches top-K passages by ISBN namespace
Chain	LangChain LCEL + PromptTemplate	Formats passages as context and submits to LLM
LLM	Llama-3.3-70b (temp = 0.1)	Generates grounded answer citing page numbers via Groq
Streaming	SSE + asyncio Queue	Delivers tokens incrementally as LLM produces them
Cache	LRU cache (256 entries)	Avoids rebuilding chain on repeated requests

4.6.2 Retriever — Similarity Threshold Filtering

The `PineconeBookRetriever` encodes the user’s question using the same `all-mpnet-base-v2` model used during ingestion — query and document vectors must occupy the same semantic space for similarity search to be meaningful. The resulting 768-dimensional query vector is submitted to Pinecone for ANN search, retrieving the top 5 candidate passages (`RAG_TOP_K = 5`) within the namespace corresponding to the requested book’s ISBN.

A similarity threshold of **0.30** is applied to the retrieved results: any passage scoring below this threshold is discarded before context assembly. This mechanism serves as a **hallucination guard** — without it, a model receiving weakly-related passages may generate plausible-sounding answers not supported by the book. If no passages exceed the threshold, the chain returns a standardised insufficient-information message.

4.6.3 Chain — Grounded Prompting and Token Budget

The RAG pipeline is implemented as a LangChain Expression Language (LCEL) chain following the “stuff” strategy, concatenating all retrieved passages into a single context block. A custom system prompt enforces three strict behavioural constraints:

- Answer using **only** the provided excerpts — no external or parametric knowledge is permitted.
- Cite the page number(s) from which each piece of information was drawn.
- Respond with a standardised message if the provided excerpts are insufficient to

answer the question.

Listing 4.2. RAG System Prompt Definition

```

1 SYSTEM_PROMPT = (
2     "You are an intelligent reading assistant for the Smart AI
      Library. "
3     "Answer using ONLY the excerpts provided below. "
4     "If the excerpts are insufficient, respond: "
5     "'I don't have enough information from this book to answer
      that.'"
6     "Always cite the page number(s) you drew information from."
7 )

```

A token budget guard truncates the assembled context to a maximum of 3,800 tokens using `tiktoken` (`cl100k_base` encoding), preventing context-overflow API failures.

4.6.4 Streaming — Server-Sent Events (SSE)

To improve perceived responsiveness, the service delivers Llama-3.3-70b-Versatile's response token-by-token via Server-Sent Events (SSE). An `asyncio Queue` acts as an in-process message bus between the LangChain streaming callback and the SSE generator coroutine. A 15-second inter-token timeout terminates the stream if the model stalls.

Table 4.8. SSE Protocol Events in the RAG Streaming Endpoint

SSE Event	Meaning
<code>data: <token></code>	A generated token — client appends it to the displayed answer
<code>data: [DONE]</code>	Generation complete — client should close the event stream
<code>data: [TIMEOUT]</code>	No token received within 15 seconds — stream terminated
<code>data: [ERROR]</code>	Exception during chain execution — stream aborted

4.7 Recommendation Service — Hybrid NCF + CBF

The Recommendation Service delivers personalised book recommendations by combining two complementary AI techniques: Neural Collaborative Filtering (NCF), which learns from user behaviour, and Content-Based Filtering (CBF), which reasons over book content semantics. A hybrid fusion layer blends the two signals with weights that adapt dynamically based on the user's interaction history.

4.7.1 Neural Collaborative Filtering (NCF)

The NCF model is a PyTorch implementation of the Generalised Matrix Factorisation + MLP architecture (He et al., 2017). It maintains two separate embedding tables for

users and items — one for the GMF branch and one for the MLP branch — allowing each pathway to learn complementary interaction patterns.

The **GMF branch** computes an element-wise product of user and item embeddings, capturing linear co-occurrence signals. The **MLP branch** concatenates the embeddings and passes them through three fully-connected layers [128, 64, 32] with ReLU activations and 0.2 dropout, capturing non-linear interaction patterns. The outputs of both branches are concatenated and passed through a final sigmoid layer to produce a relevance score in $[0, 1]$.

Table 4.9. NCF Model Hyperparameters

Hyperparameter	Value
Embedding Dimension (emb_dim)	64
MLP Hidden Layers	[128, 64, 32]
Dropout Rate	0.2
Output Activation	Sigmoid $\rightarrow$ score in $[0, 1]$
Weight Initialisation	Normal (std=0.01) for embeddings; Xavier uniform for linear layers

The model is updated incrementally via an Online Updater that consumes user interaction events from the Kafka topic `user.events`. Events classified as positive signals (`read_start`, `bookmark`, `rating_4`, `rating_5`) trigger a single SGD step on the user embedding layers only (learning rate = 5×10^{-4}), keeping item embeddings stable while personalising user representations in near real-time.

4.7.2 Content-Based Filtering (CBF)

The CBF layer uses Pinecone’s `synopses` namespace, which stores one synopsis vector per book. Each synopsis vector is the L2-normalised average of all chunk vectors for that book, computed and upserted by the Embedding Service after ingestion. To find books similar to a seed title, the CBF layer fetches the seed’s synopsis vector and queries Pinecone for the closest neighbours by cosine similarity.

4.7.3 Hybrid Fusion and User Tiers

The final recommendation score is a weighted linear blend of the NCF and CBF scores:

$$\text{score} = \alpha \times \text{CF} + (1 - \alpha) \times \text{CBF} \quad (4.1)$$

The weight α is determined by the user’s interaction tier, computed from the number of reading events recorded in the database:

Table 4.10. Hybrid Fusion Alpha Values by User Tier

Tier	Condition	Alpha (α)
New	Fewer than 5 interactions	0.10 — mostly content-based
Returning	5 to 49 interactions	0.55 — balanced blend
Power	50 or more interactions	0.80 — mostly collaborative

This tiered approach solves the cold-start problem: new users with little interaction history receive content-driven recommendations based on book semantics, while power users benefit primarily from collaborative signals learned from their accumulated behaviour. Recommendations are cached in Redis for 5 minutes per user and are subject to a rate limit of 60 requests per minute per user, enforced via a Redis sliding-window counter.

4.8 Ethical Considerations

The deployment of AI-powered features in a library context carries responsibilities beyond technical correctness. The following ethical principles were explicitly considered during the design and implementation of the platform’s AI subsystems.

4.8.1 Hallucination Mitigation

Large language models can generate factually incorrect information with high confidence — a phenomenon termed *hallucination*. In a library context, a hallucinated answer could seriously mislead researchers or students. The RAG service addresses this through two complementary mechanisms: the similarity threshold filter rejects weakly-related passages before they reach the LLM, and the system prompt explicitly forbids the model from drawing on knowledge outside the provided excerpts.

4.8.2 Transparency and Source Attribution

All RAG Q&A answers include explicit page-number citations from the source documents returned by the retriever. Users can verify any claim by consulting the original book at the cited page. This design treats AI-generated answers as a navigational aid rather than an authoritative endpoint, preserving the user’s ability to critically evaluate information.

4.8.3 Content Scope and Copyright

The platform processes only book files uploaded by authenticated administrators, who are assumed to have verified copyright compliance before upload. Generated summaries

are abstractive paraphrases intended to assist comprehension, not to substitute for reading the original work or to circumvent copyright restrictions.

4.8.4 Model Bias and Limitations

The pre-trained models used in this platform (`all-mpnet-base-v2`, `FLAN-T5-large`, `Llama-3.3-70b-Versatile`, and `cross-encoder/ms-marco-MiniLM-L-6-v2`) were not fine-tuned on library-domain data. As a result, they may perform less accurately on highly specialised technical content or non-English texts. The platform currently supports English-language content; extension to Arabic and other languages is identified as a planned future enhancement.

Chapter Summary

This chapter documented the six core AI subsystems of the Learnova platform. The Ingestion Service processes PDF and ePub files through a three-stage pipeline — text extraction, chunking, and Kafka publishing — producing traceable 512-token chunks for downstream services. The Embedding Service consumes these chunks and encodes them into L2-normalised 768-dimensional vectors using `all-mpnet-base-v2`, stored in Pinecone by ISBN namespace. The Search Service combines a bi-encoder ANN stage with a cross-encoder re-ranking stage to deliver accurate semantic search results. The Summarisation Service applies a FLAN-T5-large map-reduce pipeline to produce SHORT and DETAILED summaries and structured mind-map trees. The RAG Q&A Service combines Pinecone retrieval with Llama-3.3-70b-Versatile generation, enforcing grounded output through a similarity threshold and a strict system prompt, delivering responses via SSE streaming with page-level citations. Finally, the Recommendation Service blends NCF and CBF signals through a tiered fusion strategy, adapting dynamically to each user’s interaction history. Together, these services transform Learnova from a static digital library into an intelligent, interactive knowledge platform.

Bibliography

- [1] R. Tiwari, “The integration of AI and machine learning in education and its potential to personalize and improve student learning experiences,” *International Journal of Scientific Research in Engineering and Management*, vol. 7, no. 2, 2023.
- [2] Y. Jayaram, “AI-driven personalization 2.0: Hyper-personalized journeys for every student type,” *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, 2024.
- [3] F. L. da Silva *et al.*, “A systematic literature review on educational recommender systems,” *Education and Information Technologies*, 2023.
- [4] V. Authors, “Recommender systems: A comprehensive review (2017–2024),” *arXiv preprint*, 2024.
- [5] N. Yanes *et al.*, “A machine learning-based recommender system for improving students learning experiences,” *IEEE Access*, vol. 8, pp. 201218–201235, 2020.
- [6] Q. EL Maazouzi, A. Retbi, and S. Bennani, “Optimizing recommendation systems in E-learning: Synergistic integration of LangChain, GPT models, and retrieval augmented generation (RAG),” in *International Conference on Smart Applications and Data Analysis*, Springer Nature Switzerland, 2024.
- [7] V. Authors, “A comprehensive survey and guide to multimodal large language models in vision-language tasks,” *arXiv preprint arXiv:2411.06284*, 2024.
- [8] C. Burgan, J. Kowalski, and W. Liao, “Developing a retrieval augmented generation (RAG) chatbot app using adaptive large language models (LLM) and LangChain framework,” *Proceedings of the West Virginia Academy of Science*, vol. 96, no. 1, 2024.
- [9] I. Toniomo, “flutter_tts: A flutter plugin for text to speech.” https://pub.dev/packages/flutter_tts, 2023.
- [10] V. Authors, “Personalized E-learning recommender system based on autoencoders,” *Applied System Innovation*, vol. 6, no. 6, 2023.
- [11] V. Authors, “A digital recommendation system for personalized learning to enhance online education: A review,” *IEEE Xplore*, 2024.

- [12] U. F. Ikwuanusi, P. A. Adepoju, and C. S. Odionu, “AI-driven solutions for personalized knowledge dissemination and inclusive library user experiences,” *International Journal of Engineering Research Updates*, vol. 4, no. 2, pp. 052–062, 2023.